

前缀合 & 差分

1. 知识点：一维差分（区间修改优化）

- **核心逻辑**：通过 `D[L] += v` 和 `D[R+1] -= v` 将 $O(N)$ 的区间操作降为 $O(1)$ ，最后通过前缀和还原。
- **对应题目**：[洛谷 P2367 语文成绩](#)

2. 知识点：二维前缀和（子矩阵求和）

- **核心逻辑**：基于容斥原理，预处理 $O(N^2)$ ，查询 $O(1)$ 。
- **对应题目**：[洛谷 P1719 最大加权矩形](#)

3. 知识点：二维差分（子矩阵修改）

- **核心逻辑**：操作 4 个关键点，通过一次二维前缀和还原，将子矩阵批量修改优化至 $O(1)$ 。
- **对应题目**：[洛谷 P3397 地毯](#)

4. 知识点：差分与实际场景建模（综合应用）

- **核心逻辑**：将路线旅行转化为区间覆盖次数统计，结合贪心决策求最优解。
- **对应题目**：[洛谷 P3406 海底高铁](#)

5. 知识点：前缀和与同余定理（进阶统计）

- **核心逻辑**：利用 $(S[R] - S[L - 1]) \pmod K = 0 \Rightarrow S[R] \equiv S[L - 1] \pmod K$ 进行 $O(N)$ 优化。
- **对应题目**：[洛谷 P3131 \[USACO16JAN\] Subsequences Summing to Sevens S](#)

6. 知识点：二分答案 + 差分验证（复合应用）

- **核心逻辑**：利用答案的单调性，将“求最优解”转化为“判定可行性”。在 `check` 函数中使用差分数组 $O(N)$ 快速验证区间覆盖是否溢出。
- **对应题目**：[洛谷 P1083 \[NOIP2012 提高组\] 借教室](#)

7. 知识点：差分数组的状态平衡（逻辑深挖）

- **核心逻辑**：理解差分操作对全局状态的影响。通过统计差分数组中正数总和 P 与负数绝对值总和 Q ，最少步数为 $\max(P, Q)$ ，最终序列种数为 $|P - Q| + 1$ 。
- **对应题目**：[洛谷 P4552 \[Poetize6\] IncDec Sequence](#)

8. 知识点：异或前缀和（位运算抵消）

- **核心逻辑**：利用异或自反性 $x \oplus x = 0$ ，实现区间异或和查询： $XorSum(L, R) = S[R] \oplus S[L - 1]$ 。常用于统计出现奇偶次数或处理二进制特性。
- **对应题目**：[洛谷 P1469 找出现一次的数](#)

9. 知识点：桶统计与离散化前缀和（真题高频）

- **核心逻辑**：针对不连续的数据，先排序，再利用前缀和分别统计不同类别（如 result 为 0 或 1）的频率，从而在 $O(N)$ 内扫描出最优阈值。
- **对应题目**：[CCF-CSP 202012-2 期末预测之最佳阈值](#)（注：此为对应逻辑的 CSP 真题）

10. 知识点：状态偏移与负数下标处理

- **核心逻辑**：在处理“数量相等”或“和为 0”的问题时，前缀和可能出现负数。通过 `offset`（给下标加一个大常

数) 或 `HashMap` 记录该和第一次出现的位置。

- 对应题目: [洛谷 P2697 宝石串](#)

二分

1. 知识点: 整数二分边界 (寻找起始与终止位置)

- 核心逻辑: 区分“寻找第一个满足条件的左边界”和“最后一个满足条件的右边界”。关键在于 `mid` 的取法 (是否 `+1`) 以及 `l` 或 `r` 的更新, 防止死循环。
- 对应题目: [AcWing 789 数的范围](#) (注: 此题逻辑同洛谷 P2249)

2. 知识点: 二分答案 - 基础计数/切割模型

- 核心逻辑: 给定资源总量, 求满足需求的最大单位尺寸。 `check` 函数通常为线性累加。
- 对应题目: [洛谷 P1873 砍树](#)、[洛谷 P2440 木材加工](#)

3. 知识点: 二分答案 - 最大化最小值 (Max-Min)

- 核心逻辑: 典型的“跳石头”模型。通过二分尝试“最小距离”, 在 `check` 函数中使用贪心策略: 凡是小于 `mid` 的间隔一律通过操作 (如移走石头) 消除, 看操作次数是否超限。
- 对应题目: [洛谷 P2678 跳石头](#)

4. 知识点: 二分答案 - 最小化最大值 (Min-Max)

- 核心逻辑: 用于均衡负载或分段问题。 `check` 函数逻辑: 只要当前段累加和超过 `mid`, 就必须强制开启新段。注意左边界 `left` 应初始化为序列中的最大值。
- 对应题目: [洛谷 P1182 数列分段 Section II](#)

5. 知识点: 二分答案 - 插入类构造模型

- 核心逻辑: 在给定的不均匀间隙中插入元素, 使最大间距最小。关键点在于 `check` 函数中计算两个原始点 D 之间需补发的元素个数。
- 对应题目: [洛谷 P3853 路标设置](#)

6. 知识点: 浮点数二分 (精度控制与金融/几何建模)

- 核心逻辑: 取消 `+1/-1` 的边界处理, 使用 `l = mid` 和 `r = mid`。终止条件采用固定循环次数 (如 `for i < 100`) 以获得超越 `double` 极限的稳健精度。常用于求解方程根、月利率或几何极值。
- 对应题目: [洛谷 P1024 一元三次方程求解](#)、[洛谷 P1163 银行贷款](#)

双指针与滑动窗口

1. 知识点: 单向/对撞指针 (基础线性扫描)

- 核心逻辑: 利用序列的单调性, 通过两个指针同向或对撞移动, 将 $O(N^2)$ 的枚举优化至 $O(N)$ 。关键在于指针不回退。
- 对应题目: [洛谷 P1147 连续自然数和](#)

2. 知识点: 排序 + 多指针维护 (区间计数模型)

- 核心逻辑: 在处理 $A - B = C$ 等等值匹配时, 先排序, 再通过两个不回退的右指针 `r1, r2` 分别维护“第一

个 $\geq$ 目标值”和“第一个 $>$ 目标值”的边界，实现 $O(N)$ 统计重复元素个数。

- 对应题目：[洛谷 P1102 A-B 数对](#)

3. 知识点：变长滑动窗口（最窄覆盖模型）

- 核心逻辑：固定右端点 `r` 主动向右扩展，当窗口满足条件（如包含所有类别）时，不断收缩左端点 `l` 以寻找局部最优解。
- 对应题目：[洛谷 P1638 逛画展](#)

4. 知识点：排序映射与原序恢复（索引绑定技巧）

- 核心逻辑：当双指针匹配需要基于排序数组，但输出需遵循原始次序时，通过 `Node` 对象或 `long` 位运算绑定 `(value, original_index)`。匹配成功后利用 `boolean[]` 标记原始下标。
- 对应题目：[洛谷 P1571 眼红的 FXXZ](#)

5. 知识点：双指针预处理 + 后缀最大值（区间组合优化）

- 核心逻辑：利用双指针 $O(N)$ 预处理出每个点出发的最长合规区间，结合后缀最大值数组 `sufMax[i]`，在 $O(1)$ 时间内找到与当前区间不重叠的最优第二个区间。
- 对应题目：[洛谷 P3143 \[USACO16OPEN\] Diamond Collector S](#)

6. 知识点：数据合流与大规模滑动窗口（综合性能优化）

- 核心逻辑：针对多列表分散数据，先“合流”并按坐标排序。在 $N = 10^6$ 级别下，利用字节流 Fast I/O 和基本类型数组避开 Java 对象开销与 GC 压力。
- 对应题目：[洛谷 P2564 \[SCOI2009\] 生日礼物](#)

单调栈（Monotonic Stack）

1. 知识点：单调栈基础（寻找单侧第一个大/小）

- 核心逻辑：维护一个栈内元素单调的逻辑，通过“破坏单调性即弹出”的机制，在 $O(N)$ 时间内找到每个元素左侧或右侧第一个比其大/小的位置。栈内通常存储下标。
- 对应题目：[洛谷 P5788 【模板】单调栈](#)

2. 知识点：双向单调栈（贡献接收模型）

- 核心逻辑：通过正反两次单调栈扫描，确定每个元素左右两边最近的“更高点”。常用于模拟信号发射、雨水接纳或区间最值贡献判定。
- 对应题目：[洛谷 P1901 发射站](#)

3. 知识点：单调栈与贡献度统计（视线覆盖模型）

- 核心逻辑：在元素出栈时统计其对答案的贡献，或者利用类似 DP 的思想 `c[i] += c[index] + 1` 跳跃式累加被覆盖的区间长度。
- 对应题目：[洛谷 P2866 \[USACO06NOV\] Bad Hair Day S](#)

4. 知识点：辐射范围最大化（直方图/最小值贡献模型）

- 核心逻辑：与其枚举区间，不如固定元素 a_i 为区间最小值。利用单调栈 $O(N)$ 寻找 a_i 向左、向右能辐射到的最远边界（即左右第一个比它小的数）。结合前缀和，可在 $O(1)$ 内计算出以该元素为最小值的最大区间贡献。

- 对应题目: [洛谷 P2422 良好的感觉](#)、[洛谷 P2659 美丽的序列](#)

5. 知识点: 二维矩阵单调栈优化 (压缩直方图/悬线法思想)

- 核心逻辑: 将二维矩阵问题“降维打击”。逐行处理, 维护每一列向上连续空地的“高度” $H[j]$, 将每一行看作一个直方图的底部, 利用单调栈求解该行对应的最大矩形。最终在 $O(N \times M)$ 时间内解决二维最值问题。
- 对应题目: [洛谷 P4147 玉蟾宫](#)

单调队列 (Monotonic Queue)

1. 知识点: 滑动窗口最值 (单调队列模板)

- 核心逻辑: 使用双端队列维护下标。新元素入队前弹出队尾不如其“优秀”的元素; 获取答案前弹出队头已过期的下标 (滑出窗口)。队头始终为当前窗口最值。
- 对应题目: [洛谷 P1886 滑动窗口](#)、[洛谷 P2032 扫描](#)

2. 知识点: 单调队列优化前缀和 (限制长度的最大子段和)

- 核心逻辑: 求长度 $\leq M$ 的最大子段和即求 $Max(S[R] - S[L - 1])$, 其中 $R - (L - 1) \leq M$ 。固定 R 时, 利用单调队列维护窗口内 S 的最小值。
- 对应题目: [洛谷 P1714 切前缀和](#)

暂时跳过

位运算

1. Tricks

- `lowbit(n) = n & -n`: 获取二进制中最低位的 1 (例如: $6(0110) \rightarrow 2(0010)$)。
- 去掉最低位的 1: `n = n & (n - 1)`。例如 `n & (n-1)` 可以用来统计一个数中 1 的个数, 循环几次就是几个 1。
- 判断奇偶: `n & 1 == 1` (奇数) 或 `0` (偶数)。
- 判断 2 的幂: `(n > 0) && ((n & (n - 1)) == 0)`。
- Java 内置方法: `Integer.bitCount(n)` (统计 1 的个数)、`Integer.numberOfTrailingZeros(n)` (尾部 0 的个数)。

2. 知识点: 二进制拆分 (快速幂运算)

- 核心逻辑: 将指数 `b` 看作二进制数。利用 `b & 1` 判断当前位是否为 1, 是则乘入底数; 利用 `b >>= 1` 实现指数的“折半”处理。将时间复杂度从 $O(b)$ 优化至 $O(\log b)$ 。
- 对应题目: [洛谷 P1226 \(快速幂/取模幂\)](#)

3. 知识点: 状态压缩枚举 (子集/组合问题)

- 核心逻辑: 利用整数的二进制位 `0` 或 `1` 表示集合中元素的“不选”或“选中”。遍历 `0` 到 `(1 << N) - 1` 即可不重不漏地枚举出所有组合情况。利用 `Integer.bitCount(mask) == k` 可快速过滤出大小为 k 的合法组合。
- 对应题目: [洛谷 P1036 \(选数\)](#)

4. 知识点: 状压 DP (状态转移)

- **核心逻辑**：将整个局面编码为一个整数 `mask`。定义 `dp[mask][i]` 表示当前局面为 `mask` 且当前处于 `i` 时的最优状态。转移时使用位运算（如 `mask | (1 << j)`）将当前状态推向下一状态，外层循环必须按 `mask` 从小到大遍历以保证“无后效性”。
- **对应题目**：[洛谷 P1433 \(吃奶酪\)](#)、[洛谷 P2704 \(炮兵阵地 - 进阶\)](#)

图论

1. 知识点：DFS BFS

- **对应题目**：[P5318 【深基18.例3】查找文献](#)

// 为什么以下两种写法是错误的???

```
private static void dfs(List<Integer>[] adj, boolean[] visited, int n, int start,
    PrintWriter out) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);
    visited[start] = true;
    while(!stack.isEmpty()){
        int u = stack.pop();
        out.print(u);
        out.print(" ");

        List<Integer> vs = adj[u];
        int len = vs.size();
        for(int i = len - 1; i >= 0; i--){
            if(!visited[vs.get(i)]){
                stack.push(vs.get(i));
                visited[vs.get(i)] = true;
            }
        }
    }
}
```

```
private static void dfs(List<Integer>[] adj, boolean[] visited, int n, int start,
    PrintWriter out) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);
    visited[start] = true;
    while(!stack.isEmpty()){
        int u = stack.pop();
        out.print(u);
        out.print(" ");
        List<Integer> vs = adj[u];
        int len = vs.size();
        for(int i = len - 1; i >= 0; i--){
            if(!visited[vs.get(i)]){
                stack.push(vs.get(i));
                visited[vs.get(i)] = true;
            }
        }
    }
}
```

```

    }

    // 正确答案
    private static void dfs(List<Integer>[] adj, boolean[] visited, int n, int start,
        PrintWriter out) {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(start);
        // visited[start] = true;
        while(!stack.isEmpty()){
            int u = stack.pop();
            if(!visited[u]){
                out.print(u);
                out.print(" ");
                visited[u] = true;
            }
            List<Integer> vs = adj[u];
            int len = vs.size();
            for(int i = len - 1; i >= 0; i--){
                if(!visited[vs.get(i)]){
                    stack.push(vs.get(i));
                    // visited[vs.get(i)] = true;
                }
            }
        }
    }

    // 能区分的样例
    6 6
    1 2
    1 3
    2 4
    2 5
    4 6
    4 5

```

2. 知识点：反向建图（图论的“逆向逻辑”）

- **核心逻辑：**将图中所有的边 $u \rightarrow v$ 修改为 $v \rightarrow u$ 。
 - **为什么用它：**当原图中的“可达性”计算是**从前驱到后继**（一对多），而我们需要计算“哪些点能到达某个点”（多对一）或“**从所有点出发汇聚信息**”时，反向建图能将复杂的全局搜索简化为单次局部搜索。
 - **关键点：**反向建图不改变图的强连通分量 (SCC) 结构，但在单源路径问题和依赖传递问题中，它是打破 $O(N^2)$ 超时的唯一路径。
- **四大高频场景：**
 1. **多源汇聚（如 P3916）：**求“每个点能到达的最大编号”。原图是“从 i 出发”，反向后变成“所有能走到 i 的点”，配合从大到小的遍历，即可一次性锁定最大值。
 2. **求所有点到某个点的最短路：**Dijkstra 只能求“点 S 到所有点”。若题目要求“所有点到点 S 的距离”，直接在反向图上从 S 跑一次 Dijkstra，结果与原图等价。
 3. **强连通分量 (SCC) 的 Kosaraju 算法：**该算法的核心步骤就是通过“反向建图 + 二次 DFS”来划分强连通分量。
 4. **博弈论或依赖消除：**当一个状态 A 依赖于后继状态 B 和 C 时，如果在原图上难以按顺序计算，反向建图可以将依赖链反转，变为从已知状态向未知状态的“推导”。

- 对应题目：
 - [洛谷 P3916 \(图的遍历\)](#) 这题不赖
 - [洛谷 P1629 \(邮递员送信 - 最短路应用\)](#) (此题非常经典，要求所有点到某点的往返，利用反向建图可以极大地减少计算量)

3. 知识点：Dijkstra 算法

- **核心逻辑**：基于贪心策略，每次选取当前距离源点最近的节点进行松弛。使用 `PriorityQueue` 优化后，复杂度为 $O(M \log N)$ 。在 Java 实现中，必须使用 `long[]` 存储距离以防止 `Integer.MAX_VALUE` 相加导致的溢出。**优化关键**：在 `while` 循环中加入 `if (dist[u] < d) continue;` 进行惰性删除，避免无效计算。
- 对应题目：[洛谷 P4779 \(单源最短路径-标准版\)](#)

4. 知识点：0-1 BFS (双端队列最短路)

- **核心逻辑**：当边权仅为 0 或 1 时，无需使用 `PriorityQueue`。利用 `ArrayDeque` 实现双端操作：权为 0 的边入队头，权为 1 的边入队尾。复杂度优化为 $O(V + E)$ ，是解决网格图、矩阵搜索类问题的“降维打击”算法。
- 对应题目：[洛谷 P4667 \(Switch the Lamp\)](#)

5. 知识点：Floyd-Warshall 算法 (任意两点间最短路)

- **核心逻辑**：基于动态规划，`dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`。外层循环必须是“中转点 `k`”，内层循环为 `i` 和 `j`。
- **适用场景**： $N \leq 500$ 的稠密图，或题目要求查询任意两点间距离。
- **CSP 陷阱**：时间复杂度 $O(N^3)$ 。禁止在 $N \geq 1000$ 的题目中使用，且注意 `dist[i][k] + dist[k][j]` 加法操作可能导致的溢出。
- 实现：

```
public static void floydWarshall(int[][] graph, int n) {
    // 创建距离矩阵，初始为图的邻接矩阵副本
    int[][] dist = new int[n][n];
    for (int i = 0; i < n; i++) {
        System.arraycopy(graph[i], 0, dist[i], 0, n);
    }

    // 核心算法：三重循环
    // k 作为中间顶点
    for (int k = 0; k < n; k++) {
        // i 作为起点
        for (int i = 0; i < n; i++) {
            // j 作为终点
            for (int j = 0; j < n; j++) {
                // 如果经过 k 的路径比当前路径更短，则更新
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    // 输出最终的最短距离矩阵
    printSolution(dist, n);
}
```

```
}
```

- 对应题目: [洛谷 P2910 \(旅行路线\)](#)

6. 知识点: 分层图最短路

- **核心逻辑:** 当路径搜索带有“机会次数 K ”或“状态切换”限制时, 构建 $N \times (K + 1)$ 个节点的新图。每一层代表一次状态改变, 在层间通过 0 权边实现跳转。本质是状态空间的图构建。
- 对应题目: [洛谷 P4568 \(飞行路线\)](#)

7. 知识点: Bellman-Ford 算法

- **核心逻辑:** 基于动态规划 (状态转移) 思想, 核心动作是对所有边进行“松弛”。算法的底层原理在于其**逐层递推**的过程: 外层循环的每一轮 K ($1 \leq K \leq N - 1$), 其真实的物理含义是——**计算出从起点出发, 最多经过 K 条边到达各个顶点的最短路径**。因为在一个包含 N 个顶点的图中, 任意两点间的最短简单路径 (不兜圈子) 最多只包含 $N - 1$ 条边, 所以强制执行 $N - 1$ 轮遍历, 就能像水波一样把最短路径的信息推导至极限。这种“按边的跳数全局复盘”的机制, 彻底摒弃了 Dijkstra 的局部贪心, 因此能完美处理**负权边**。进一步地, 如果进入第 N 轮遍历 (即允许走 N 条边) 时依然能发生松弛, 根据鸽巢原理, 走 N 条边必然重复经过了某个顶点, 且总权重还在缩小, 这必定意味着图中存在**负权环** (无限倒贴钱的死循环)。时间复杂度为 $O(N \times M)$ 。**优化关键:** 在 $N - 1$ 轮循环中加入 `boolean updated;` 标记, 若第 K 轮没有任何距离被更新, 说明“最多经过 K 条边”的结果已经是最优解, 全局最短路已提前确立, 直接 `break` 结束循环, 这是避免 TLE 的黄金剪枝策略。
- 实现:

```
/**
 * Bellman-Ford 算法核心实现
 * @param n 顶点个数
 * @param edges 边集, 每条边为 {u, v, w}
 * @param source 源点
 * @return dist 数组, 若包含负权环则返回 null
 */
public static int[] bellmanFord(int n, int[][] edges, int source) {
    int INF = Integer.MAX_VALUE / 2;
    int[] dist = new int[n];
    Arrays.fill(dist, INF);
    dist[source] = 0;

    // 松弛 n-1 次
    for (int i = 1; i < n; i++) {
        boolean updated = false;
        for (int[] e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] < INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                updated = true;
            }
        }
        if (!updated) break;
    }

    // 检测负权环
    for (int[] e : edges) {
        int u = e[0], v = e[1], w = e[2];
```



```

        if (dist[u] < INF && dist[u] + w < dist[v]) {
            return null; // 存在负权环
        }
    }

    return dist;
}

```

- 对应题目: [洛谷 P3385 【模板】负环](#)

8. 知识点: 并查集 (DSU)

- **核心逻辑:** 通过路径压缩优化查询效率, 利用树形结构管理不相交集合, 实现连通性判断及动态连通块维护, 是处理复杂图论合并问题的利器。
- 对应题目:
 - [洛谷 P3367 【模板】并查集](#)
 - [洛谷 P1551 亲戚](#)
 - [洛谷 P1197 \[JSOI2008\] 星球大战](#)

拓扑排序

```

public static int[] topologicalSortKahn(int n, List<Integer>[] graph) {
    int[] inDegree = new int[n];

    // 计算入度
    for (int u = 0; u < n; u++) {
        for (int v : graph[u]) {
            inDegree[v]++;
        }
    }

    // 将入度为0的顶点入队
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) queue.offer(i);
    }

    int[] result = new int[n];
    int index = 0;

    while (!queue.isEmpty()) {
        int u = queue.poll();
        result[index++] = u;

        for (int v : graph[u]) {
            if (--inDegree[v] == 0) {
                queue.offer(v);
            }
        }
    }
}

```

```
// 检查是否有环
return index == n ? result : null;
}
```

动态规划

线性 DP

1. 知识点：最大子段和与降维打击

- **核心逻辑**：一维最大子段和状态转移为 $dp[i] = \max(dp[i-1] + a[i], a[i])$ 。在二维矩阵中，通过外层两层循环固定上边界和下边界，利用前缀和将二维列压缩成一维数组，从而将二维矩阵求和降维成一维最大子段和，避免 $O(N^4)$ 暴力。
- **对应题目**：[洛谷 P1115 最大子段和](#)、[洛谷 P1719 最大加权矩形](#)

2. 知识点：最长上升子序列 (LIS) 及其 $O(N \log N)$ 优化

- **核心逻辑**：原始 $O(N^2)$ 做法遍历寻找 $dp[j]$ 。优化做法**转换“状态”与“值”**，定义 $g[len]$ 表示长度为 len 的上升子序列中**最小的结尾值**。 g 数组天然单调递增，遍历新元素时，利用二分查找在 g 中找到第一个 $\geq$ 该元素的数并替换之。
- **对应题目**：[洛谷 P1020 \[NOIP1999 普及组\] 导弹拦截](#)

3. 知识点：最长公共子序列 (LCS) 转化为 LIS

- **核心逻辑**：当两个序列是无重复元素的排列时，LCS 可以通过映射法则降维打击为 LIS。将序列 A 作为标准参考系（元素映射为其下标），将同样的映射规则应用到序列 B 上，对转换后的序列 B 求 $O(N \log N)$ 的 LIS 即为答案。
- **对应题目**：[洛谷 P1439 【模板】最长公共子序列](#)

4. 知识点：逆向推导 DP（打破无后效性限制）

- **核心逻辑**：当当前的选择会“锁死”未来的时间点（正向推导失去无后效性）时，改变时间之矢，**从后往前倒着 DP**。站在第 i 个时间点看，状态依赖于做完任务后的第 $i + T$ 个时间点的状态： $dp[i] = \max(dp[i+T]) + val$ 。
- **对应题目**：[洛谷 P1280 尼克的任务](#)

5. 知识点：双向 LIS 拼接（山峰模型）

- **核心逻辑**：处理“递增后递减”的合唱队形/山峰形结构。正难则反，不求剔除最少，求留下最多。枚举每个点作为山峰，预处理出从左往右的 LIS ($f[i]$) 和从右往左的 LIS ($g[i]$)，答案即为 $\max(f[i] + g[i] - 1)$ 。
- **对应题目**：[洛谷 P1091 \[NOIP2004 提高组\] 合唱队形](#)

6. 知识点：DP 的路径记录

- **核心逻辑**：DP 不仅求最值，还需输出具体方案。在进行状态转移 $dp[i] = dp[j] + val$ 且取得更优解时，同步维护一个数组 $pre[i] = j$ ，记录是从哪个状态转移而来的。最后从全局最优的终点出发，利用 pre 数组逆向回溯出整条路径。
- **对应题目**：[洛谷 P2196 \[NOIP1996 提高组\] 挖地雷](#)

7. 知识点：隐式图的拓扑排序 DP（消除二维网格后效性）

- **核心逻辑**：对于只能往某一极值方向（如从高到底）移动的二维网格，直接循环无法进行 DP。将所有网格点作为一个 Node 提取出来，根据限制条件（如高度）进行**全局排序**。排序后的顺序即为有向无环图（DAG）的**拓扑序**，按照此顺序依次进行 DP 转移即可实现 $O(NM \log(NM))$ 。
- **对应题目**：[洛谷 P1434 \[SHOI2002\] 滑雪](#)

8. 知识点：带属性维度的状态空间压缩

- **核心逻辑**：当状态转移需要依赖特定属性（如等差数列的“公差”）时，标准做法是开二维数组 `dp[i][d]`。但当属性范围极大且物品数量不多时，**将属性提至最外层循环固定**，内部降维至一维数组 `cnt[v]` 表示“当前固定公差下，以数值 v 结尾的方案数”。利用 `cnt[v] += cnt[prev] + 1` 实现平行分支树的完美累加，极大降低空间复杂度。
- **对应题目**：[洛谷 P4933 洛谷团队编年史/大师](#)

9. 知识点：线段跳跃 DP + 二分查找优化

- **核心逻辑**：用于解决区间不重叠调度问题。按区间**右端点**从小到大排序。状态定义为 `dp[i]` 表示前 i 个线段能取得的最大收益。转移方程：`dp[i] = max(dp[i-1], dp[last_valid] + val)`。由于区间按右端点有序，利用二分查找快速寻找“最后一个右端点小于当前左端点的线段” `last_valid`，将复杂度由 $O(N^2)$ 降为 $O(N \log N)$ 。
- **对应题目**：[洛谷 P1868 饥饿的奶牛](#)

0-1 背包

完全背包

完全背包问题和 0-1 背包问题非常相似，**区别仅在于不限制物品的选择次数**。

- 在 0-1 背包问题中，每种物品只有一个，因此将物品 i 放入背包后，只能从前 $i - 1$ 个物品中选择。
- 在完全背包问题中，每种物品的数量是无限的，因此将物品 i 放入背包后，**仍可以从前 i 个物品中选择**。

在完全背包问题的规定下，状态 $[i, c]$ 的变化分为两种情况。

- **不放入物品 i** ：与 0-1 背包问题相同，转移至 $[i - 1, c]$ 。
- **放入物品 i** ：与 0-1 背包问题不同，转移至 $[i, c - wgt[i - 1]]$ 。

从而状态转移方程变为：

$$dp[i, c] = \max(dp[i - 1, c], dp[i, c - wgt[i - 1]] + val[i - 1])$$

1. 知识点：多重背包与“二进制拆分” (Binary Splitting)

- **核心逻辑**：物品有限制数量 C_i 。直接展开的时间复杂度为 $O(V \times \sum C_i)$ ，极其缓慢。利用二进制思想，将 C_i 个物品打包成大小为 $1, 2, 4, 8 \dots (C_i - 2^k + 1)$ 的 $\log C_i$ 个新包裹。因为这些包裹可以拼凑出 $0 \sim C_i$ 之间的任意取法，所以直接将这新包裹作为全新物品，跑一遍基础的 0-1 背包即可，复杂度暴降至 $O(V \times \sum \log C_i)$ 。
- **对应题目**：[洛谷 P1776 宝物筛选](#)

2. 知识点：分组背包与“极危循环序”

- **核心逻辑**：物品被划分为 K 组，每组内的物品互斥，最多只能选一个。
-  **致命易错点（三重循环顺序必须死记）**：
 1. 最外层：枚举不同的**分组** k 。
 2. 中间层：枚举背包**容量** c （**必须逆序！**）。
 3. 最内层：枚举当前分组内的**物品** i 。
 注：若将容量和物品循环颠倒，会导致同一个组内的物品被重复放入背包。
- **对应题目**：[洛谷 P1757 通天之分组背包](#)

3. 知识点：依赖背包与组合降维 (OOP 思想)

- **核心逻辑**：购买附件必须先买主件。若使用传统的 `if-else` 枚举附件情况，代码会极其冗长且难以扩展。
- **高级技巧**：将“主件”及其所有“附件”的搭配方案（例如：仅主件、主+附1、主+附2、主+附1+附2）视为一个**分组内的互斥物品**。通过面向对象 (OOP) 建立 `Group` 类，利用递归或位运算自动生成该主件所有的合法组合（幂集），从而极其优雅地将“**依赖背包**”转化为“**分组背包**”解决。
- **对应题目**：[洛谷 P1064 \[NOIP2006 提高组\] 金明的预算方案](#)

提醒点

- 在涉及同余的题目中，请养成使用 $(\text{sum} \% 7 + 7) \% 7$ 的习惯，因为java中对负数取余会产生负余数
- `S[i] / mid - (S[i] % mid == 0 && S[i] != 0 ? 1 : 0);` 和 `(S[i] - 1) / mid;` 是一样的
- 处理 10^5 的数据的时候，最好不要使用 `HashMap`，内存可能会炸
- **内存优化技巧 (long 压缩)**：若需存储两个 `int`（如坐标和类型、值和索引）并进行排序，可使用 `long` 数组替代对象数组。通过 `(long)v << 32 | (id & 0xFFFFFFFLL)` 压缩，`Arrays.sort(long[])` 比对象排序更快且内存占用仅为 1/3 到 1/4。
- 数组容量心算预估：128MB 限制下 `int[]` 的安全上限约 $1.5 * 10^8 \sim 2 * 10^8$
- 8 (`long[]` 减半，256MB 翻倍)，请时刻预留 30MB 左右的 JVM 基础开销。
- 警惕 Java 内存刺客（防 MLE）：① 杜绝包装类（`Integer[]` 占用是 `int[]` 的 5 倍以上）；② 二维数组对象头开销大，行数极多且内存紧张时，务必用一维数组模拟二维（`arr[i * cols + j]`）。
- 清空栈的代价：在多组数据或多次扫描时，使用 `top = -1` 即可实现 $O(1)$ 清空手写栈，严禁使用 $O(N)$ 的 `Arrays.fill(arr, 0)`，因为反正把 `top` 改成了 -1 之后也会覆盖。
- 记得 `ans` 的初始化可能会导致某些边界出错，比如下面的代码有什么问题？

```
public static void main(String[] args){
    FastReader fr = new FastReader(System.in);
    PrintWriter out = new PrintWriter(System.out);

    int a = fr.nextInt(), b = fr.nextInt(), p = fr.nextInt(), bb = b;
    long ans = 1, pow = a;
    while(b > 0){
        if((b & 0x1) == 1){
            ans *= pow;
            ans %= p;
        }
        b >>= 1;
        pow = pow * pow % p;
    }
    out.println(ans);
}
```

```

    }
    pow *= pow;
    pow %= p;
    b >>= 1;
}

out.printf("%d^%d mod %d=%d", a, bb, p, ans);

out.flush();
out.close();
}

```

- 小数学

- GCD

```

int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

```

- LCM

```

long lcm(int a, int b) {
    return (long)a / gcd(a, b) * b; // 先除后乘防止溢出
}

```

- 判定素数

- 单个

```

bool isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}

```

- 线性筛 (找到1 - N的所有素数, O(N))

```

int primes[N], cnt = 0;
bool st[N]; // st[i] 为 true 表示 i 是合数
void get_primes(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) primes[cnt++] = i;
        for (int j = 0; primes[j] * i <= n; j++) {
            st[primes[j] * i] = true;
            if (i % primes[j] == 0) break; // 保证每个合数只被最小质因子筛掉
        }
    }
}

```

- 在环上找上一个数和下一个数, 下标为 1 ~ n:

```
int left = i % n + 1; // 顺时针
int right = (i - 2 + n) % n + 1; // 逆时针
```

- 求并集时可以做个小优化：

```
private static HashSet<Integer> AndSet(HashSet<Integer> integers, HashSet<Integer>
integers1) {
    HashSet<Integer> ans = new HashSet<>();
    // 性能小技巧：遍历小的集合，在大的集合里查 contains
    if (integers.size() > integers1.size()) {
        HashSet<Integer> temp = integers; integers = integers1; integers1 = temp;
    }
    for(int val : integers){
        if(integers1.contains(val)) ans.add(val);
    }
    return ans;
}
```